

Лабораторна робота № 6 Хеш-таблиці

Мета: Застосувати структуру даних хеш-таблиця та виконати основні операції

Теоретичні відомості

Таблиці з прямою адресацією

Хеш-таблиця представляє собою узагальнення звичайного масиву. Можливість прямої індексації елементів звичайного масиву дозволяє доступ до довільної позиції в масиві за час $O(1)$.

Загалом пряма індексація представляє собою технологію, яка добре працює для невеликих множин ключів. Припустимо, що застосуванню потрібна динамічна множина, кожен елемент якої має ключ з множини $U = \{0, 1, \dots, m-1\}$, де m не дуже велике. Крім того, припускається, що жодні два елементи не мають однакових ключів.

Для представлення динамічної множини використовується масив, або таблиця з прямою адресацією, який позначається як $T[0 \dots m-1]$, кожна позиція, або комірка, якого відповідає ключу з простору ключів U . На рис. 1 представлений цей підхід. Комірка k вказує на елемент множини з ключем k . Якщо множина не містить елемента з таким ключем, то $T[k] = \text{NIL}$. На рисунку кожен ключ простору $U = \{0, 1, \dots, 9\}$ відповідає індексу таблиці. Множина реальних ключів $K = \{2, 3, 5, 8\}$ визначає комірки таблиці, які містять покажчики на елементи. Решта комірок містять значення NIL .

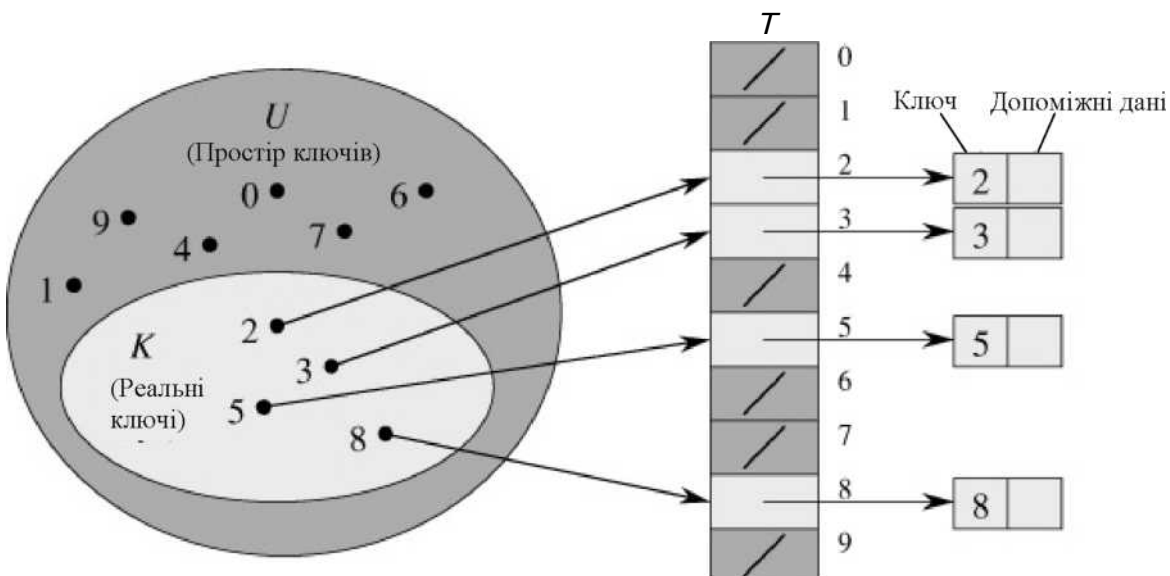


Рис. 1 Таблиця з прямою адресацією

Реалізація основних операцій в масиві тривіальна.

DirectAddressSearch(T, k)

1 **return** $T[k]$

DirectAddressInsert(T, x)

1 $T[\text{key}[x]] \leftarrow x$

DirectAddressDelete(T, x)

1 $T[key[x]] \leftarrow NIL$

Лістинг 1. Процедури, які реалізують операції роботи з масивами.

Кожна з наведених операцій дуже швидка: час їх роботи дорівнює $O(1)$.

В деяких застосуваннях елементи динамічної множини можуть зберігатись безпосередньо в таблиці з прямою адресацією. Тобто замість зберігання ключів та допоміжних даних елементів в об'єктах, які є зовнішніми по відношенню до таблиці з прямою адресацією, а в таблиці - показчиків на ці об'єкти, ці об'єкти можна зберігати безпосередньо в комірках таблиці (що призводить до економії пам'яті).

Хеш-таблиці

Недолік прямої адресації очевидний: якщо простір ключів U великий, то зберігання таблиці T розміром $|U|$ непрактично, а для деяких задач навіть неможливо. Крім того, множина K реальних ключів може бути малою по відношенню до U , а в цьому випадку пам'ять, яка виділена під T , здебільшого витрачається дарма.

Коли множина K ключів, які зберігаються у словнику, значно менша простору U , хеш-таблиця вимагає значно менше місця, чим таблиця з прямою адресацією. Точніше кажучи, вимоги до пам'яті можуть бути знижені до $O(|K|)$, при цьому час пошуку елементу в хеш-таблиці лишається рівним $O(1)$. Потрібно тільки відзначити, що це межа середнього часу пошуку, в той час як у випадку таблиці з прямою адресацією ця межа справедлива для найгіршого випадку.

У випадку прямої адресації елемент з ключем k зберігається у комірці k . При хешуванні цей елемент зберігається в комірці $H(k)$, тобто тут використовується **хеш-функція** H для обчислення комірки для даного ключа k . Функція H відображає простір ключів U на комірки **хеш-таблиці** $T[0..t-1]$:

$$H: U \rightarrow \{0, 1, \dots, t-1\}.$$

Ми говоримо, що елементи з ключем k хешуються в комірку $H(k)$; величина $H(k)$ називається хеш-значенням ключа k . На рис. 2 представлена основна ідея хешування. Мета хеш-функції полягає в тому, щоб зменшити робочий діапазон індексів масиву, а замість $|U|$ значень ми можемо обійтись лише t значеннями. Відповідно знижуються вимоги до об'єму пам'яті.

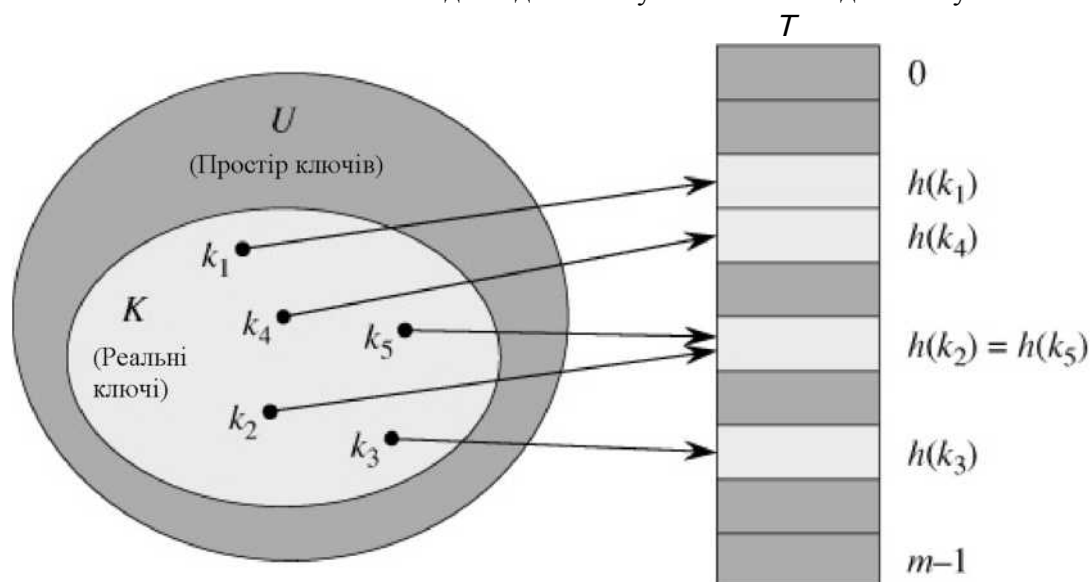


Рис. 2. Використання хеш-функції для відображення ключів у комірки хеш-таблиці.

Проте тут є одна проблема: два ключа можуть мати одне й те саме хеш-значення. Так ситуація називається **колізією**. На щастя є ефективні технології для розв'язання конфліктів, які

виникають під час колізій.

Звісно, ідеальним рішенням було би повне уникнення колізій. Цього можна досягнути шляхом підбора відповідної хеш-функції. Одна з ідей полягає в тому, щоб зробити функцію H випадковою. Зрозуміло, що хеш-функція мусить бути детермінованою і для одного й того самого значення k завжди давати одне й те саме значення $H(k)$. Однак оскільки $|U| > t$, повинні існувати як мінімум два ключа, які мають однакове хеш-значення. Таким чином, повністю уникнути колізій принципово неможливо, і добра хеш-функція може тільки мінімізувати кількість колізій.

Уникнення колізій за допомогою ланцюгів

За допомогою методу ланцюгів всі елементи, які хешуються в одну й ту саму комірку, об'єднуються у зв'язаний список, як це показано на рис. 3. Комірка j містить показчик на заголовок списку всіх елементів, хеш-значення ключа яких дорівнює j ; якщо таких елементів немає, комірка містить значення NIL. На рис. 3 наведено розв'язання колізій, які виникають через те, що $h(k_1) = h(k_4)$, $h(k_5) = h(k_2) = h(k_7)$ та $h(k_3) = h(k_8)$.

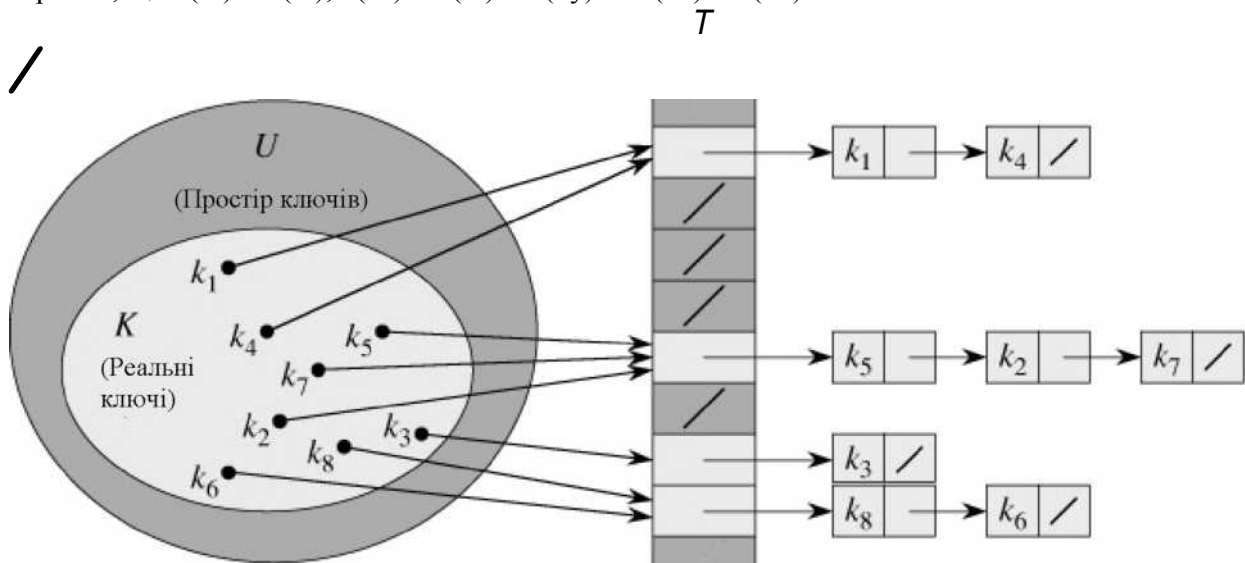


Рис. 3. Розв'язання колізій за допомогою ланцюгів.

Словарні операції в хеш-таблиці із використанням ланцюгів для розв'язання колізій реалізуються дуже просто:

ChainedHashInsert(T, x)

1 Вставити x в заголовок списку $T[h(key[x])]$

ChainedHashSearch(T, k)

1 Пошук елемента з ключем k в списку $T[h(k)]$

ChainedHashDelete(T, x)

1 Видалення x зі списку $T[h(key[x])]$

Лістинг 8.2. Процедури, які реалізують операції роботи з хеш-таблицями.

Час, необхідний для вставки в найгіршому випадку, дорівнює $O(1)$. Процедура вставки виконується дуже швидко, адже передбачається, що елемент для вставки відсутній у таблиці. При необхідності це припущення може бути перевірено виконанням пошуку перед вставкою. Час роботи пошуку в найгіршому випадку пропорційний довжині списку. Видалення елемента може бути виконане за час $O(1)$ при використанні двозв'язаних списків.

Наскільки висока ефективність хешування з ланцюгами? Зокрема, скільки часу потрібно для пошуку елемента з даним ключем?

Нехай ми маємо хеш-таблицю T з t комірок, в яких зберігаються p елементів. Визначимо коефіцієнт заповнення таблиці T як $a = p/t$, тобто як середню кількість елементів, які зберігаються в одному ланцюгу. Подальший аналіз буде ґрунтуватись на значенні величини a , яка може бути менше, дорівнювати або більше одиниці.

В найгіршому випадку хешування з ланцюгами поводить себе досить неприємно: всі p ключів хешуються в одну й ту саму комірку, створюючи при цьому список довжиною p . Таким чином, час пошуку в найгіршому випадку дорівнює $O(p)$ плюс час обрахунку хеш-функції, що нічим не краще використання зв'язаного списку для збереження всіх p елементів. Зрозуміло, що використання хеш-таблиць в найгіршому випадку немає сенсу.

Середня продуктивність хешування залежить від того, наскільки добре хеш-функція h розподіляє множину ключів для зберігання по t комірок в середньому. Ми розглянемо це питання детальніше згодом, а зараз припустимо, що всі елементи хешуються по комітках рівномірно та незалежно, і назвемо це припущення «простим рівномірним хешуванням».

Результати аналізу складності операцій хеш-таблиць. Якщо кількість комірок в хеш-таблиці, як мінімум, пропорційна кількості елементів, які зберігаються в ній, то $p = O(m)$ і, відповідно, $a = p/m = O(m)/m = O(1)$, і значить пошук елементу в хеш-таблиці в середньому потребує сталий час. Оскільки в найгіршому випадку вставка елементу в хеш-таблиці займає $O(1)$ часу (як і видалення елементу при використанні двонаправлених списків), можна зробити висновок, що всі словникові операції в хеш-таблиці в середньому виконуються за час $O(1)$.

Хеш-функції

Якісна хеш-функція задовольняє (наближено) припущення простого рівномірного хешування: для кожного ключа рівно ймовірно розміщення в будь-яку з t комірок, незалежно від хешування інших ключів. Нажаль, цю умову зазвичай неможливо перевірити, оскільки, як правило, розподілення ймовірностей, згідно з яким поступають нові ключі, невідоме; крім того, вставлені ключі можуть й не бути незалежними.

Іноді розподілення ймовірностей виявляється відомим. Наприклад, якщо відомо, що ключі представляють собою випадкові дійсні числа, які рівномірно розподілені в діапазоні $0 < k < 1$, то хеш-функція $h(k) = [km]$ задовольняє умові простого рівномірного хешування.

На практиці при побудові якісних хеш-функцій часто використовується різні евристичні методики. В процесі побудови значно допомагає інформація про розподіл ключів. Розглянемо, наприклад, таблицю символів компілятора, в якій ключами слугують символні рядки, які представляють ідентифікатори в програмі. Часто в одній програмі зустрічаються схожі ідентифікатори, наприклад, `pt` та `pts`. Якісна хеш-функція повинна мінімізувати шанси потрапляння цих ідентифікаторів в одну комірку хеш-таблиці.

При побудові хеш-функції гарним підходом є підбір функції таким чином, щоб вона ніяк не корелювала із закономірностями, яким можуть підпорядковуватись існуючі дані. Також деякі застосування хеш-функцій можуть накладати додаткові вимоги, окрім вимог простого рівномірного хешування. Наприклад, можна вимагати, щоб «близькі» в деякому сенсі ключі давали далекі хеш-значення.

Для більшості хеш-функцій простір ключів представляється множиною невід'ємних цілих чисел $N = \{0, 1, 2, \dots\}$. Якщо ж ключі не є цілими невід'ємними числами, то можна знайти спосіб їх інтерпретації як таких. Наприклад, рядок символів може розглядатись як ціле число, яке записане у відповідній системі числення. Так, ідентифікатор `pt` можна розглядати як пару десятичних чисел (112, 116), оскільки в ASCII-наборі символів `p` = 112 та `t` = 116. Розглядаючи `pt` як число в системі числення з основою 128, ми бачимо, що воно відповідає значенню $112 \cdot 128 + 116 = 14452$. В конкретних застосуваннях зазвичай не виявляється надто

складно розробити метод для представлення ключів у вигляді цілих (можливо, великих) чисел. Далі ми будемо вважати, що всі ключі представлені як цілі невід’ємні числа.

Розглянемо наступні два методи побудови хеш-функцій: метод ділення та метод множення.

Побудова хеш-функції **методом ділення** полягає у відображенні ключа k в одну з комірок шляхом отримання остачі від ділення k на m , тобто хеш-функція має вигляд $h(k) = k \bmod m$. Наприклад, якщо хеш-таблиця має розмір $m = 12$, а значення ключа $k = 100$, то $h(k) = 4$. Оскільки для обчислення хеш-функції потрібна тільки одна операція ділення, хешування методом ділення вважається достатньо швидким.

При використанні даного методу зазвичай намагаються уникнути деяких значень m . Наприклад, m не повинно бути степенем 2, оскільки якщо $m = 2^r$, то $h(k)$ представляє собою просто r молодших бітів числа k . Якщо тільки завчасно не відомо, що всі набори молодших r бітів ключів рівно ймовірні, краще будувати хеш-функції таким чином, щоб її результат залежав від усіх бітів ключа.

Часто добрі результати можна отримати, якщо обирати в якості значення m просте число, достатньо далеке від степеня двійки. Припустимо, наприклад, що ми хочемо створити хеш-таблицю із розв’язком колізій методом ланцюгів для збереження $n = 2000$ символьних рядків, розмір символів в яких дорівнює 8 бітам. Нас влаштовує перевірка у середньому трьох елементів при невдалому пошуку, так що ми обираємо розмір таблиці рівним $m = 701$. Число 701 обрано як просте, яке близьке до $2000/3$.

Побудова хеш-функції **методом множення** виконується в два етапи. Спочатку ключ k помножується на сталу $0 < A < 1$ і береться дробова частина отриманого добутку. Потім це значення помножується на m і результат замінюється на найближче менше ціле число,

$$h(k) = \lfloor m (kA \bmod 1) \rfloor,$$

тобто:

де вираз “ $kA \bmod 1$ ” означає отримання дробової частини добутку kA .

Перевага методу множення полягає в тому, що значення m перестає бути критичним. Зазвичай величина m із міркувань зручності реалізації функції обирається рівною степеню 2. Нехай ми маємо комп’ютер із розміром слова w бітів та k розміщується в одне слово. Обмежимо можливі значення константи A виглядом $S/2^w$, де S - ціле число з діапазону $0 < S < 2^w$. Тоді ми спочатку помножимо k на w -бітове ціле число $s = A \cdot 2^w$. Результат представляє собою $2w$ -бітове число $Tj2^w + r_0$, де r_1 - старше слово добутку, а r_0 - молодше. Старші p бітів числа r_0 представляють собою шукане p -бітове хеш-значення (рис. 4).

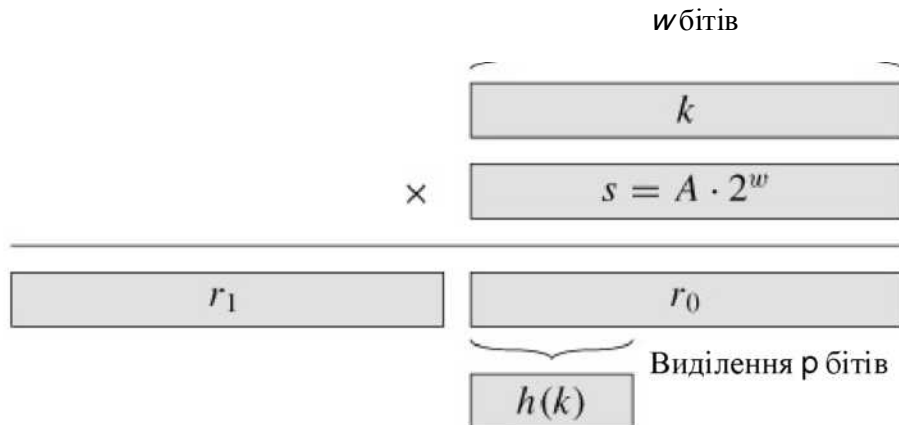


Рис. 4. Хешування методом множення.

Хоч описаний метод працює з довільними значеннями константи A , деякі значення

дають кращі результати у порівнянні з іншими. Оптимальний вибір залежить від характеристик хешуємих даних. Так, непогані результати дає значення

$A \gg (\sqrt{5} - 1)/2 \gg 0,6180339887$ к Наприклад, при $k = 123456$, $p = 14$,
 $t = 2^{14} = 16384$ та $w = 32$, оберемо значення A у вигляді 2^w , яке найближче до $(\sqrt{5} - 1)/2$, так що $A = 2654435769/2^{32}$. Тоді

$k \cdot 3 = 327\,706\,022\,297\,664 = (76\,300 \cdot 2^{32}) + 17\,612\,864$, і, відповідно, $G_i = 76\,300$ та $G_0 = 17\,612\,864$. Старші 14 бітів числа G_0 дають хеш-значення $H(k) = 67$.

У наведених способах побудови хеш-функцій є один суттєвий недолік. Якщо злоумисник буде свідомо обирати ключі для хешування за допомогою конкретної хеш-функції, то він зможе підібрати p значень, які будуть хешуватись в одну й ту саму комірку таблиці, що призведе до середнього часу вибірки $O(p)$. Таким чином, будь-яка фіксована хеш-функція стає вразливою, і єдиний ефективний вихід з цієї ситуації - випадковий вибір хеш-функції, який не залежить від того, з якими саме ключами їй доведеться працювати. Такий підхід, який називається **універсальним хешуванням**, гарантує добру якість в середньому, не залежно від того, які дані будуть обрані злоумисниками.

Головна ідея універсального хешування полягає у випадковому виборі хеш-функції з деякого ретельно відібраного класу функцій на початку роботи програми. Як і у випадку швидкого сортування, рандомізація гарантує, що одні й ті самі вхідні дані не можуть постійно давати найгіршу поведінку алгоритму. В силу рандомізації алгоритм буде працювати кожен раз по-різному, навіть для одних й тих самих вхідних даних, що гарантує високу середню продуктивність. Для більше детальної інформації по універсальному хешуванню рекомендуємо звернутись до літератури, наведеної у кінці даного матеріалу.

Відкрита адресація

При використанні методу відкритої адресації всі елементи зберігаються безпосередньо в хеш-таблиці, тобто кожний запис таблиці містить або елемент динамічної множини, або значення NIL. При пошуку елемента ми систематично перевіряємо комірки таблиці до тих пір, доки не знайдемо шуканий елемент або поки не переконаємось у його відсутності в таблиці. Тут, на противагу методу ланцюгів, немає ані списків, ані елементів, які зберігаються поза таблицею. Таким чином, в методі відкритої адресації хеш-таблиця може виявитись заповненою, що унеможливить додавання нових елементів; коефіцієнт заповнення a не може бути більшим за 1.

Для виконання вставки при відкритій адресації ми послідовно перевіряємо комірки хеш-таблиці до тих пір, доки не знайдемо порожню комірку, в яку розміщується новий ключ. Замість фіксованого порядку дослідження комірок $0, 1, \dots, m - 1$ (для чого потрібний час $O(m)$), послідовність досліджуваних комірок залежить від ключа, який вставляється у таблицю. Для визначення досліджуваних комірок хеш-функція розширюється, включаючи в якості другого аргументу номер дослідження (який починається з 0).

результати хеш-функція стає наступною:
В $h: U \times \{0, 1, \dots, m - 1\} \rightarrow \{0, 1, \dots, m - 1\}$.

У методі відкритої адресації потрібно, що для кожного ключа k послідовність досліджень $\langle h(k, 0), h(k, 1), \dots, h(k, m - 1) \rangle$

представляла собою перестановку множини $\{0, 1, \dots, m - 1\}$, щоб в кінцевому випадку можна було переглянути всі комірки хеш-таблиці. У наведеному нижче псевдокоді припускається, що елементи в таблиці T представляють собою ключі без додаткової інформації; ключ k тотожний елементу, який містить ключ k . Кожна комірка містить або ключ, або значення NIL (якщо вона не заповнена):

HashInsert(T, k)

```

1 i ← 0
2 repeat j ← h(k, i)
3   if T[j] = NIL
4     then T[j] ← k
5     return j
6   else i ← i + 1
7 until i = m
8 error "Хеш-таблиця переповнена"

```

Лістинг 3. Процедура додавання елементу до відкритої адресації.

Алгоритм пошуку ключа k досліджує ту саму послідовність комірок, що алгоритм вставки. Таким чином, якщо при пошуку зустрічається порожня комірка, пошук завершується невдало, оскільки ключ k повинен був бути вставлений в цю комірку у послідовності досліджень, але ніяк не після неї. Процедура HashSearch отримує в якості вхідних параметрів хеш-таблицю T та ключ k і повертає номер комірки, яка містить ключ k (або значення NIL, якщо ключ в хеш-таблиці не знайдений):

HashSearch(T, k)

```

1 i ← 0
2 repeat j ← h(k, i)
3   if T[j] = k
4     then return j
5   i ← i + 1
6 until T[j] = NIL або i = m
7 return NIL

```

Лістинг 4. Процедура пошуку ключа у відкритій адресації.

Процедура видалення з хеш-таблиці із відкритою адресацією достатньо складна. При видаленні ключа з комірки I ми не можемо просто помітити її значенням NIL. Зробивши так, ми можемо зробити неможливим вибірку ключа k , в процесі вставки якого досліджувалась і виявилась зайнятою комірка I . Одне з рішень полягає в тому, щоб помічати такі комірки спеціальним значенням DELETED замість NIL. При цьому ми повинні трохи змінити процедуру HashInsert, з тим щоб вона розглядала таку комірку як порожню та могла вставляти в неї новий ключ. В процедурі HashSearch жодних змін робити не потрібно, оскільки ми просто пропускаємо такі комірки при пошуку та досліджуємо наступні комірки під час пошуку в послідовності. Однак при використанні спеціального значення DELETED час пошуку перестає залежати від коефіцієнту заповнення α і тому, як правило, при необхідності видалення з хеш-таблиці в якості методу розв'язання колізій обирається метод ланцюгів.

У подальшому ми будемо виходити із припущення **рівномірного хешування**, тобто ми припускаємо, що для кожного ключа в якості послідовності досліджень рівноймовірні всі $m!$ перестановок множини $\{0, 1, \dots, m-1\}$. Рівномірне хешування представляє собою узагальнення визначеного раніше простого рівномірного хешування, яка полягає в тому, що тепер хеш-функція дає не одне значення, а цілу послідовність досліджень. Реалізація істинно рівномірного хешування достатньо важка, але на практиці використовуються прийнятні апроксимації.

Для обчислення послідовності досліджень для відкритої адресації зазвичай використовуються три методи: лінійне дослідження, квадратичне дослідження та подвійне хешування. Ці методи гарантують, що для кожного ключа k ($h(k,0), h(k,1), \dots, h(k, m-1)$) є перестановкою $\{0, 1, \dots, m-1\}$. Одна ці методи не задовольняють припущенню про рівномірне хешування, адже жоден з них не в змозі згенерувати більше m^2 різних послідовностей (замість $m!$, які потрібні для рівномірного хешування). Найбільша кількість послідовностей досліджень дає подвійне хешування і, як слід, найкращі результати.

Нехай задана звичайна хеш-функція $h': U \rightarrow \{0, 1, \dots, m-1\}$, яку будемо надалі йменувати **допоміжною хеш-функцією**. Метод лінійного дослідження для обчислення послідовності

досліджень використовує хеш-функцію

$$h(k, i) = (h'(k) + i) \bmod m, \text{ де } i \text{ приймає значення від } 0 \text{ до } m - 1.$$

Для заданого ключа k першою досліджуваною коміркою є $T[h'(k)]$, тобто комірка, яку дає допоміжна хеш-функція. Далі досліджуються комірки $T[h'(k) + 1]$, $T[h'(k) + 2]$, ..., $T[m - 1]$, а потім переходять до $T[0]$, $T[1]$, і далі до $T[h'(k) - 1]$. Оскільки початкова досліджувана комірка однозначно визначає всю послідовність досліджень в цілому, всього є m різних послідовностей.

Лінійне дослідження легко реалізується, однак з ним пов'язана проблема первинної кластеризації, яка полягає у створенні довгих послідовностей зайнятих комірок, що, саме по собі, збільшує середній час пошуку. Кластери виникають через те, що ймовірність заповнення порожньої комірки, якій передують і заповнених комірок, дорівнює $(1 + 1)/m$. Таким чином, довгі серії заповнених комірок мають тенденцію до все більшого подовження, що призводить до збільшення середнього часу пошуку.

Квадратичне дослідження використовує хеш-функцію вигляду

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m,$$

де h' - допоміжна хеш-функція, c_1 та $c_2 \neq 0$ - допоміжні константи, а i приймає значення від 0 до $m - 1$. Початкова досліджувана комірка - $T[h'(k)]$; решта досліджуваних позицій зміщені відносно неї на величину, які описуються квадратичною залежністю від номеру дослідження i . Цей метод працює значно краще лінійного дослідження, але для того, щоб дослідження охопило всі комірки, необхідний вибір спеціальних значень c_1 , c_2 та m . Окрім того, якщо два ключі мають одну й ту саму початкову позицію дослідження, то однакові й послідовності досліджень у цілому, адже з $h_1(k, 0) = h_2(k, 0)$ слідує $h_1(k, i) = h_2(k, i)$. Ця властивість приводить до більш м'якої вторинної кластеризації. Як і у випадку лінійного дослідження, початкова комірка визначає всю послідовність, тому всього використовуються m різних послідовностей досліджень.

Подвійне хешування представляє собою один з найкращих методів використання відкритої адресації, оскільки отримані при цьому перестановки мають багато характеристик випадкового згенерованих перестановок. Подвійне хешування використовує хеш-функцію наступного вигляду:

$$h(k, i) = (h_1(k) + i h_2(k)) \bmod m, \text{ де } h_1 \text{ та } h_2 - \text{допоміжні хеш-функції.}$$

Початкове дослідження виконується в позиції $T[h_1(k)]$, а зміщення кожної з наступних досліджуваних комірок відносно попередньої дорівнює $h_2(k)$ по модулю m . Відповідно, на відміну від лінійного та квадратичного дослідження, в даному випадку послідовність досліджень залежить від ключа k по двох параметрах - в плані вибору початкової досліджуваної комірки та відстані між сусідніми досліджуваними комірками. На рис. 5 показаний приклад вставки при подвійному хешуванні. Тут наведена хеш-таблиця розміром 13 комірок, в які використовуються допоміжні хеш-функції $h_1(k) = k \bmod 13$ та $h_2(k) = 1 + (k \bmod 11)$. Так як $14 \equiv 1 \pmod{13}$ та $14 \equiv 3 \pmod{11}$, ключ 14 вставляється в порожню комірку 9, після того як при дослідженні комірок 1 та 5 з'ясується, що ці комірки зайняті.

Для того щоб послідовність досліджень могла охопити всю таблицю, значення $h_2(k)$ повинно бути взаємно простим із розміром хеш-таблиці m . Зручний спосіб забезпечити виконання цієї умови полягає у виборі числа m , рівного ступеню 2, і розробці хеш-функції h_2 таким чином, щоб вона повертала тільки непарні значення. Ще один спосіб полягає у використанні в якості m простого числа і побудові хеш-функції h_2 такою, щоб вона завжди повертала натуральні числа, менші m . Наприклад, можна обрати просте число в якості m , а хеш-функції такими:

0	
1	79
2	
3	
4	69
5	98
6	
7	72
8	
9	14
10	
11	50
12	

Рис. 5. Вставка елемента при подвійному хешуванні.

де m' повинно бути трохи менше m

$$\begin{aligned}h_1(k) &= k \bmod m, \\h_2(k) &= 1 + (k \bmod m'),\end{aligned}$$

(наприклад, $m = 701$). скажімо, якщо $k = 123456$, $m = 701$, а $m' = 700$, то $h_1(k) = 80$ та $h_2(k) = 257$, так що першою досліджуваною коміркою буде 80-а комірка, а потім буде досліджуватись кожна 257-а (за модулем m) комірка, доки не буде знайдена порожня комірка, або доки не виявиться що досліджені всі комірки таблиці.

Подвійне хешування краще лінійного та квадратичного дослідження в сенсі кількості $O(m^2)$ послідовностей досліджень. Це пов'язано з тим, що кожна можлива пара $(h_1(k), h_2(k))$ дає свою, відмінну від інших послідовність досліджень. У результаті продуктивність подвійного хешування достатньо близька до продуктивності «ідеальної» схеми рівномірного хешування.

Аналіз відкритої адресації, як і аналіз методу ланцюгів, виконується із використанням коефіцієнту заповнення $a = n/m$ хеш-таблиці при n та m , які прямують до нескінченності. Зрозуміло, при використанні відкритої адресації $a \leq 1$, адже не може бути більше одного елемента на комірку таблиці ($n \leq m$).

Якщо хеш-таблиця заповнена наполовину, очікувана кількість досліджень при вдалому пошуку не більша за 1,387; при наповненості в 90% ця величина не перевищує 2,559.

При закритому хешуванні хеш-таблиця є масивом, елементи якого занумеровані від 0 до $m-1$.

При відкритому хешуванні хеш-таблиця є масивом, кожна комірка якого містить покажчики на заголовок списку всіх елементів, хеш-значення ключа яких дорівнює індексу комірки.

1. Побудова хеш-функції методом ділення

Ідея

Значенням хеш-функції є остача від ділення ключа на розмір хеш-таблиці.

Псевдокод

$$h(x) = x \bmod m$$

2. Пошук елемента при відкритому хешуванні

Ідея

Знайти відповідне хеш-значення і здійснити пошук у списку, заголовок на який міститься у відповідній комірці хеш-таблиці.

Псевдокод

```
заголовок = хеш-таблиця [h (ключ)]  
знайти позицію ключа в списку, заданого заголовком  
повернути позиція != NULL
```

3.Запис елемента при відкритому хешуванні

Ідея

Якщо елемент не знайдено, то додати його на початку списку, що відповідає хеш-значенню.

Псевдокод

```
якщо Пошук (ключ) == FALSE то  
{  
    заголовок = хеш-таблиця [h (ключ)]  
    вставити ключ в позицію 1 списку, заданого заголовком  
}
```

4.Вилучення елемента при відкритому хешуванні

Ідея

Вилучити елемент зі списку, заголовок на який міститься у відповідній комірці хеш-таблиці.

Псевдокод

```
заголовок = хеш-таблиця [h (ключ)]  
p = позиція ключа в списку, заданого заголовком  
видалити елемент в позиції p зі списку, заданого заголовком
```

5. Вставка елемента при закритому хешуванні

Ідея

При лінійному хешуванні використовується хеш-функція $h(x, i) = (h_1(x) + i) \bmod m$, де $h_1(x)$ — звичайна хеш-функція, i — номер спроби розмістити елемент. Якщо комірка, відповідна хеш-значенню зайнята, то обчислюється значення для наступної спроби.

Псевдокод

```
i = 0  
повторювати  
{  
    k
```

```
= h(x, i)
}
поки хеш-таблиця[k] == 0
хеш-таблиця[k] = x
```

Завдання для виконання

Постановка задачі

Скласти програми, що забезпечують виконання операцій з хеш-таблицями. Програми повинні забезпечувати

- реалізацію основних процедур із хеш-таблицею;
- заповнення хеш-таблиці послідовністю випадково згенерованих чисел;
- виведення на екран графа хеш-таблиці;
- виконання вказаних викладачем дій з хеш-таблицею.

Технічні умови

- розмір хеш-таблиці дорівнює 13;
- множина ключів містить 50 елементів.

Хід роботи

1. Скласти програму Hash.cpp для реалізації операцій з хеш-таблицею при відкритому хешуванні; хеш-функція будується методом ділення.
2. Забезпечити виконання дій, зазначених викладачем.
3. Виконати програму в покроковому режимі.
4. Скласти програму Hash2.cpp для реалізації додавання елементів у хеш-таблицю при лінійному хешуванні і заповнення її випадково згенерованими числами.
5. Виконати програму в покроковому режимі для 8 випадкових чисел.

Запитання для самоконтролю

1. У чому особливість хеш-таблиці?
2. У чому відмінність відкритого і закритого хешування?
3. Поясніть принципи виконання основних операцій при відкритому хешуванні.
4. Поясніть побудову хеш-функції при лінійному хешуванні.

Література

Cormen, Thomas H.; Leiserson, Charles E., Rivest, Ronald L., Stein, Clifford (2001) [1990]. Introduction to Algorithms (2nd ed.). MIT Press and McGraw-Hill. ISBN 0-262-03293-7. Г лава 11.
Лабораторні роботи Хеш таблиці. goo.gl/KS9uGA